

راهنمای جامع شبیه‌سازی آونگ دوتایی و آشوب معین روی موبایل

آزمایشگاه محاسباتی در جیب — اپیزود ۱

۱. مقدمه و پیش‌زمینه علمی

در فیزیک کلاسیک نیوتنی، جهانی که ما می‌شناسیم بر پایه‌ی جبر و قطعیت بنا شده است: اگر موقعیت و سرعت اولیه‌ی تمام ذرات یک سیستم را بدانیم، می‌توانیم سرنوشت آن را تا ابد محاسبه کنیم. این نگرش مکانیکی قرن‌ها پارادایم حاکم بر علم بود؛ تا زمانی که مفهوم آشوب معین (**Deterministic Chaos**) این فرضیه را به چالش کشید.

سیستم‌های آشوبناک تصادفی (**Random**) نیستند؛ در این سیستم‌ها شانس یا احتمالات کوانتومی هیچ نقشی ندارد و حرکت آن‌ها به‌طور دقیق با معادلات قطعی ریاضی بیان می‌شود. با این حال، رفتار سیستم در گذر زمان به گونه‌ای تغییر می‌کند که در عمل غیرقابل پیش‌بینی است.

آونگ دوتایی چیست و چرا آشوبناک است؟

یک آونگ ساده‌ی تکی دارای حرکت نوسانی هارمونیک با دوره‌ی تناوب معین است. اما اگر بازوی دوم را با یک لولای آزاد به انتهای بازوی اول متصل کنیم، یک آونگ دوتایی (**Double Pendulum**) شکل می‌گیرد. در این پیکربندی:

- حرکت بازوی اول مستقیماً به موقعیت و سرعت زاویه‌ای بازوی دوم نیرو، شتاب و گشتاور وارد می‌کند و برعکس.

- این کوپلینگ (جفت‌شدگی) نیروها باعث می‌شود معادلات دیفرانسیل سیستم به شدت غیرخطی (**Non-linear**) شوند.

- در این سامانه هیچ راه‌حل تحلیلی بسته (**Closed-form Analytical Solution**) وجود ندارد؛ یعنی نمی‌توان فرمولی روی کاغذ نوشت که با قرار دادن زمان t در آن، زاویه‌ی آونگ‌ها را مستقیماً به دست آورد. تنها راه شناخت آینده، حل عددی گام‌به‌گام معادلات است.

اثر پروانه‌ای و حساسیت شدید به شرایط اولیه

ویژگی بنیادین سامانه‌های آشوبناک، واگرایی نمایی در فضای فاز (Exponential Divergence) است. اگر دو سیستم کاملاً یکسان را با تفاوت اولیه‌ی بی‌نهایت ناچیز (برای مثال تنها یک درجه یا حتی 0.001 رادیان) همزمان رها کنیم:

۱. در گام‌های اول، هر دو سیستم رفتار مشابهی از خود نشان می‌دهند.

۲. با گذر زمان، تفاضل زاویه‌ها با ضریب مشخصی موسوم به نمای لیاپانوف (Lyapunov Exponent) به صورت نمایی رشد می‌کند:

$$\Delta\theta(t) \sim \Delta\theta_0 e^{\lambda t}$$

۳. پس از عبور از یک بازه زمانی معین، شباهت دو مسیر به صفر رسیده و سیستم‌ها وارد دو سرنوشت کاملاً نامربوط به یکدیگر می‌شوند. این پدیده دلیل قطعی ناتوانی ابرکامپیوترها در پیش‌بینی دقیق وضعیت جو و هواشناسی در بازه‌های زمانی بیش از دو هفته است.

۲. مدل‌سازی ریاضی و حل عددی

استخراج معادلات لاگرانژ

برای توصیف حرکت آونگ دوتایی با طول بازوهای L_1, L_2 و جرم‌های متمرکز M_1, M_2 ، از مکانیک لاگرانژی استفاده می‌شود. با انتخاب زاویه‌های θ_1 و θ_2 نسبت به راستای قائم به عنوان متغیرهای تعمیم‌یافته:

موقعیت دکارتی جرم اول:

$$x_1 = L_1 \sin \theta_1, \quad y_1 = -L_1 \cos \theta_1$$

موقعیت دکارتی جرم دوم:

$$x_2 = x_1 + L_2 \sin \theta_2, \quad y_2 = y_1 - L_2 \cos \theta_2$$

لاگرانژین سیستم برابر است با تفاضل انرژی جنبشی (T) از انرژی پتانسیل (V):

$$\mathcal{L} = T - V$$

با اعمال معادلات اوایلر-لاگرانژ برای هر دو زاویه:

پوریا آزادی

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\theta}_i} \right) - \frac{\partial \mathcal{L}}{\partial \theta_i} = 0 \quad (i = 1, 2)$$

دستگاه معادلات دیفرانسیل مرتبه دوم برای شتاب‌های زاویه‌ای ($\ddot{\theta}_1 = \alpha_1$ و $\ddot{\theta}_2 = \alpha_2$) بر حسب زوایا و سرعت‌های زاویه‌ای ($\dot{\theta}_1 = \omega_1$ و $\dot{\theta}_2 = \omega_2$) استخراج می‌شود:

$$\alpha_1 = \frac{-g(2M_1 + M_2) \sin \theta_1 - M_2 g \sin(\theta_1 - 2\theta_2) - 2 \sin(\theta_1 - \theta_2) M_2 (\omega_2^2 L_2 + \omega_1^2 L_1 \cos(\theta_1 - \theta_2))}{L_1 (2M_1 + M_2 - M_2 \cos(2\theta_1 - 2\theta_2))}$$

$$\alpha_2 = \frac{2 \sin(\theta_1 - \theta_2) (\omega_1^2 L_1 (M_1 + M_2) + g(M_1 + M_2) \cos \theta_1 + \omega_2^2 L_2 M_2 \cos(\theta_1 - \theta_2))}{L_2 (2M_1 + M_2 - M_2 \cos(2\theta_1 - 2\theta_2))}$$

حل عددی با روش رانژ-کوتا مرتبه ۴ (RK4)

چون حل تحلیلی ناممکن است، سیستم چهار معادله‌ی مرتبه اول زیر را تشکیل می‌دهد:

$$\mathbf{y} = \begin{bmatrix} \theta_1 \\ \omega_1 \\ \theta_2 \\ \omega_2 \end{bmatrix}, \quad \frac{d\mathbf{y}}{dt} = \begin{bmatrix} \omega_1 \\ \alpha_1(\mathbf{y}) \\ \omega_2 \\ \alpha_2(\mathbf{y}) \end{bmatrix}$$

برای جلوگیری از خطای عددی و پایستار ماندن انرژی در شبیه‌سازی درازمدت، از روش رونگه-کوتا مرتبه چهار (Runge-Kutta 4th Order) استفاده شده است. همچنین به جای جهش‌های بزرگ زمانی، تکنیک زیرگام محاسباتی (Sub-stepping) به کار گرفته شده تا در هر فریم انیمیشن، معادلات طی ۵ مرحله ریز محاسباتی حل شوند:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

۳. پیش‌نیازها و راه‌اندازی بستر روی گوشی

اجرای این شبیه‌سازی به سخت‌افزار قدرتمند یا کامپیوتر نیازی ندارد؛ پردازنده‌ی تلفن همراه با راه‌اندازی ابزارهای متن‌باز زیر برای رندر خروجی کافی است:

مسیر اصلی: اجرای بومی در ترموکس (Android)

در صورتی که طبق [آموزش پیشین در کانال](#)، لینوکس اوبونتو را نصب کرده‌اید، وارد اوبونتو شوید:

```
Bash
```

```
proot-distro login ubuntu
```

سپس پیش‌نیازهای محاسباتی و پکیج ساخت فایل گیف را با دستور زیر در اوبونتو نصب کنید:

```
Bash
```

```
apt update && apt install python3 python3-numpy python3-matplotlib python3-pil -y
```

مسیر جایگزین: گوگل کولب (از طریق مرورگر)

کاربران آیفون که نمیتوانند ترموکس را نصب کنند یا کسانی که قصد اجرای شبیه‌سازی ساده‌تر در مرورگر را دارند، می‌توانند به آدرس colab.research.google.com مراجعه کرده، یک Notebook جدید باز کنند و کد زیر را در سلول اول قرار داده و اجرا نمایند.

۴. کد کامل پایتون (Python Implementation)

اسکرپت زیر شبیه‌سازی را برای دقیقاً ۲۰ ثانیه (۵۰۰ فریم با نرخ ۲۵ فریم بر ثانیه) محاسبه کرده و دو آونگ را که زاویه اولیه‌ی آنها دقیقاً ۱ درجه (1.0°) با یکدیگر اختلاف دارد، روی یک نمودار عمودی ۹:۱۶ با رنگ‌های نئونی متضاد و دنباله‌های دنباله‌دار نقاشی می‌کند:

```
Python
```

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation, PillowWriter

# =====
# 1. PHYSICAL CONSTANTS & EQUATIONS OF MOTION
# =====
G = 9.81      # Acceleration due to gravity (m/s^2)
L1, L2 = 1.0, 1.0 # Pendulum rod lengths (meters)
M1, M2 = 1.0, 1.0 # Bob masses (kg)

def derivatives(state):
    """
    Computes angular accelerations using coupled Lagrangian mechanics.
    state = [theta1, omega1, theta2, omega2]
```

```

"""
t1, w1, t2, w2 = state
delta = t1 - t2
den = 2 * M1 + M2 - M2 * np.cos(2 * t1 - 2 * t2)

# Angular acceleration for bob 1 (alpha1)
num1 = (-G * (2 * M1 + M2) * np.sin(t1)
        - M2 * G * np.sin(t1 - 2 * t2)
        - 2 * np.sin(delta) * M2 * (w2**2 * L2 + w1**2 * L1 * np.cos(delta)))
alpha1 = num1 / (L1 * den)

# Angular acceleration for bob 2 (alpha2)
num2 = (2 * np.sin(delta) * (w1**2 * L1 * (M1 + M2)
        + G * (M1 + M2) * np.cos(t1)
        + w2**2 * L2 * M2 * np.cos(delta)))
alpha2 = num2 / (L2 * den)

return np.array([w1, alpha1, w2, alpha2])

def rk4_step(state, dt):
    """4th-order Runge-Kutta numerical integration step."""
    k1 = derivatives(state)
    k2 = derivatives(state + 0.5 * dt * k1)
    k3 = derivatives(state + 0.5 * dt * k2)
    k4 = derivatives(state + dt * k3)
    return state + (dt / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)

# =====
# 2. SIMULATION TIMING & INITIAL PERTURBATION
# =====
FPS = 25
DURATION_SEC = 20          # Exact 20 seconds duration
TOTAL_FRAMES = DURATION_SEC * FPS # Exactly 500 frames
DT_FRAME = 1.0 / FPS
SUB_STEPS = 5              # 5 steps per frame for high numerical stability
DT_SUB = DT_FRAME / SUB_STEPS

# Initial angles: Released from horizontal position (90 degrees)
INITIAL_THETA1 = np.pi / 2
INITIAL_THETA2 = np.pi / 2

# Exact 1.0 degree perturbation for system B
DELTA_THETA = np.radians(1.0)

state_a = np.array([INITIAL_THETA1, 0.0, INITIAL_THETA2, 0.0])
state_b = np.array([INITIAL_THETA1 + DELTA_THETA, 0.0, INITIAL_THETA2, 0.0])

history_a = np.zeros((TOTAL_FRAMES, 4))
history_b = np.zeros((TOTAL_FRAMES, 4))

```

```

print(f"Integrating physics for {DURATION_SEC}s ({TOTAL_FRAMES} frames)...")
for frame in range(TOTAL_FRAMES):
    history_a[frame] = state_a
    history_b[frame] = state_b
    for _ in range(SUB_STEPS):
        state_a = rk4_step(state_a, DT_SUB)
        state_b = rk4_step(state_b, DT_SUB)

# Convert polar angles to Cartesian coordinates (X, Y)
# System A (Cyan)
x1_a = L1 * np.sin(history_a[:, 0])
y1_a = -L1 * np.cos(history_a[:, 0])
x2_a = x1_a + L2 * np.sin(history_a[:, 2])
y2_a = y1_a - L2 * np.cos(history_a[:, 2])

# System B (Magenta)
x1_b = L1 * np.sin(history_b[:, 0])
y1_b = -L1 * np.cos(history_b[:, 0])
x2_b = x1_b + L2 * np.sin(history_b[:, 2])
y2_b = y1_b - L2 * np.cos(history_b[:, 2])

# =====
# 3. VISUALIZATION (VERTICAL 9:16 FORMAT)
# =====
fig, ax = plt.subplots(figsize=(6, 10.6), facecolor="#020202")
ax.set_facecolor("#020202")
ax.set_xlim(-2.3, 2.3)
ax.set_ylim(-2.5, 1.5)
ax.axis("off")

# Fixed center anchor
ax.plot(0, 0, marker="o", markersize=6, color="FFFFFF", alpha=0.9)

# System A (Cyan - Base Pendulum)
trail_a, = ax.plot([], [], color="#00E5FF", lw=1.6, alpha=0.75, label="Base Pendulum")
line_a, = ax.plot([], [], "o-", color="#80F0FF", lw=2.2, markersize=7, zorder=4)

# System B (Neon Magenta - Perturbed by 1.0 Degree)
trail_b, = ax.plot([], [], color="#FF0077", lw=1.6, alpha=0.75, label="Perturbed (+1.0°)")
line_b, = ax.plot([], [], "o-", color="#FFA0CF", lw=2.2, markersize=7, zorder=5)

# Digital HUD overlay metrics
time_text = ax.text(0.05, 0.94, "", transform=ax.transAxes, color="FFFFFF",
                    fontsize=12, family="monospace", weight="bold")
delta_text = ax.text(0.05, 0.90, "Delta Theta: 1.0°",
                    transform=ax.transAxes, color="FFD700", fontsize=10, family="monospace")

# High-contrast legend
ax.legend(loc="upper right", facecolor="#101010", edgecolor="#2B2B2B",
        labelcolor="E0E0E0", fontsize=9, framealpha=0.85)

```

```
TRAIL_LENGTH = 100 # 4 seconds of dynamic neon trajectory history

def update(frame):
    start_idx = max(0, frame - TRAIL_LENGTH)

    # Update neon trails
    trail_a.set_data(x2_a[start_idx:frame], y2_a[start_idx:frame])
    trail_b.set_data(x2_b[start_idx:frame], y2_b[start_idx:frame])

    # Update physical arms
    line_a.set_data([0, x1_a[frame], x2_a[frame]], [0, y1_a[frame], y2_a[frame]])
    line_b.set_data([0, x1_b[frame], x2_b[frame]], [0, y1_b[frame], y2_b[frame]])

    # Update real-time clock
    current_time = frame * DT_FRAME
    time_text.set_text(f"Time: {current_time:.1f}s / {DURATION_SEC}.0s")

    return trail_a, trail_b, line_a, line_b, time_text

ani = FuncAnimation(fig, update, frames=TOTAL_FRAMES, interval=1000/FPS, blit=True)

# =====
# 4. EXPORT ANIMATION
# =====
output_filename = "double_pendulum_chaos_20s.gif"
print(f"Rendering {TOTAL_FRAMES} frames ({DURATION_SEC}s)...")
ani.save(output_filename, writer=PillowWriter(fps=FPS))
print(f"Done! File saved successfully as: {output_filename}")
```

۵. راهنمای گام به گام اجرا و خروجی گرفتن روی گوشی

۱. ایجاد فایل کد:

در محیط ترمینال لینوکس اوبونتو، یک فایل متنی پایتون بسازید:

Bash

nano double_pendulum.py

۲. ذخیره سازی:

کد بالا را کپی کرده و داخل ترمینال پیست کنید. سپس کلیدهای **Ctrl + O** (ذخیره) و پس از آن **Enter** و در نهایت **Ctrl + X** (خروج) را بزنید.

۳. اجرا:

دستور زیر را اجرا کنید:

```
Bash
```

```
python3 double_pendulum.py
```

پردازنده‌ی گوشی ابتدا ۵۰۰ فریم فیزیک را در کسری از ثانیه انتگرال‌گیری عددی می‌کند و سپس نوار رندر انیمیشن آغاز می‌شود. پس از تکمیل، پیام Done! روی صفحه ظاهر خواهد شد.

۴. انتقال به گالری موبایل:

برای اینکه فایل double_pendulum_chaos_20s.gif را در گالری گوشی تماشا کنید، آن را به حافظه اصلی گوشی منتقل کنید:

```
Bash
```

```
# خارج شدن از اوبونتو به محیط اصلی ترموکس
```

```
exit
```

```
# گوشی Download کپی از حافظه اوبونتو به پوشه
```

```
cp ~/ubuntu/root/double_pendulum_chaos_20s.gif /sdcard/Download/
```

اکنون وارد پوشه‌ی Download در فایل منیجر یا گالری گوشی شوید و انیمیشن را پخش کنید.

۶. تحلیل فیزیکی خروجی شبیه‌سازی

پس از باز کردن انیمیشن، این سه مرحله‌ی رفتاری را به دقت زیر نظر بگیرید:

[0.0s - 3.5s] [3.5s - 5.0s] [5.0s - 20.0s]

آشوب مهارناپذیر (Break) نقطه دوشاخگی انطباق کامل

(دو مدار کاملاً متضاد) --> (شروع باز شدن زاویه‌ها) --> (خط سفید مشترک)

فاز انطباق (ثانیه ۰ تا ۳٫۵):

با وجود اختلاف ۱ درجه در نقطه رهاسازی، دو آونگ کاملاً همگام حرکت می‌کنند؛ رنگ فیروزه‌ای و سرخابی دقیقاً روی هم منطبق بوده و یک نور ترکیبی را نشان می‌دهند.

نقطه انشعاب یا دوشاخگی (Bifurcation Point):

در حدود ثانیه ۳۰۸، یکی از وزنه‌ها انرژی جنبشی لازم برای گذر از نقطه‌ی تعادل اوج را کسب می‌کند اما دیگری به اندازه‌ی یک کسر ژول کم می‌آورد.

فاز آشوب نهایی (ثانیه ۵ تا ۲۰):

ناگهان یکی از آونگ‌ها یک دور کامل ۳۶۰ درجه می‌زند در حالی که دیگری در پایین نوسان معکوس می‌کند. پس از ثانیه ۶، هیچ بیننده‌ای بدون دانستن تاریخچه‌ی سیستم نمی‌تواند حدس بزند که نقطه‌ی شروع این دو ساختار، زاویه‌ای یکسان بوده است.

۷. آزمایش‌ها و چالش‌های محاسباتی برای علاقه‌مندان

برای کشف ابعاد عمیق‌تر این پدیده، کدهای شبیه‌سازی را باز کرده و پارامترها را تغییر دهید:

چالش ۱: تست مرز حساسیت لیپانوف

مقدار DELTA_THETA را به جای ۱ درجه، برابر با 0.0001 رادیان بگذارید:

```
Python
```

```
DELTA_THETA = 0.0001
```

شبیه‌سازی را دوباره ران کنید. ثبت کنید که لحظه‌ی جدایی مسیرها دقیقاً از ثانیه چندم آغاز می‌شود.

چالش ۲: شکستن تقارن جرم‌ها

جرم آونگ دوم را ده برابر کنید ($M2 = 10.0$). بررسی کنید که آیا اثر شلاقی بازوی انتهایی، سیستم را سریع‌تر وارد فاز آشوب می‌کند یا دیرتر؟

چالش ۳: تولید اثر نفونی ماندگار (Long Exposure)

پارامتر TRAIL_LENGTH را از ۱۰۰ به ۵۰۰ تغییر دهید تا تمام مسیرهای پیموده شده از ابتدا تا انتهای ۲۰ ثانیه روی تصویر باقی بمانند و یک اثر هنری پیچیده‌ی هندسی خلق شود.

برای دنبال کردن باقی آموزش‌ها وارد کانال تلگرام زیر شوید.

https://t.me/the_maze2022